UNIT II

IMPLEMENTING POLYMORPHISM IN C ++

Polymorphism

- The word **polymorphism** is derived from Greek words meaning **“many forms.”** It is one of the important concepts of **Object-Oriented Programming (OOP)**.
- polymorphism means that **the same function name or operator can perform different operations depending on the context, object, or data type involved**.

Types of Polymorphism

- Polymorphism is mainly divided into two types

I. **Compile-Time Polymorphism:** The function or operator is decided at the time of compilation based on how it is called.

II. **Run-Time Polymorphism:** The function is decided while the program is running based on the object being used.

Types of Polymorphism

POLYMORPHISM

The ability of the same interface or function to exhibit different behaviors in different situations.

COMPILE TIME POLYMORPHISM

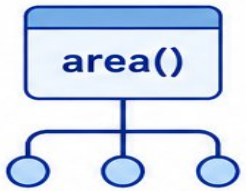
(Static Binding)

Resolved at compile time.

1. FUNCTION OVERLOADING

Same function name with different parameter lists.

```
void area(int r);
void area(double r);
void area(int l, int b);
```



2. OPERATOR OVERLOADING

Same operator works differently for user-defined data types.

```
Complex c1, c2, c3;
c3 = c1 + c2;   // '+' is
 overloaded
```



RUN TIME POLYMORPHISM

(Dynamic Binding)

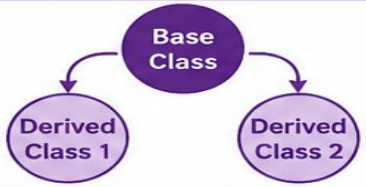
Resolved at run time.

3. FUNCTION OVERRIDING

Same function name in base and derived class with different implementation.

Implemented using **Virtual Functions.**

```
class Base {
    virtual void display();
};
class Derived : public Base {
    void display();   // overridden
};
```



Advantage: Polymorphism allows code reusability, flexibility and extensibility by using a single interface for different forms (behaviors).

Compile-Time Polymorphism

- Compile-time polymorphism is also known as static polymorphism or early binding. In this type of polymorphism, the compiler decides which function or operator to call during compilation. It is achieved using:

I. **Function Overloading**

II. **Operator Overloading**

Compile-Time Polymorphism

I. Function Overloading

Function overloading allows multiple functions to have the same name but different parameters list. The difference can be in:

- Number of parameters
- Type of parameters
- The overloaded functions perform different tasks depending on the **number or type of arguments** supplied during the function call.
- It is also called **function polymorphism** and is an example of **compile-time polymorphism (static polymorphism)**.

Compile-Time Polymorphism

I. Function Overloading

Syntax:

return_type function_name(parameter_list);

Example:

```
int add(int, int);
int add(int, int, int);
double add(double, double);
```

All functions have the same name add(), but their argument lists are different.

Compile-Time Polymorphism

I. Function Overloading Need

- Function overloading provides:
  - Code reusability
  - Better readability
  - Simpler program design
  - Ability to perform related operations using a single function name
  - Support for compile-time polymorphism

Example

```
#include <iostream>
using namespace std;

// Function declarations (prototypes)
int volume(int);
double volume(double, int);
long volume(long, int, int);

int main()
{
    cout << volume(10) << "\n";
    cout << volume(2.5, 8) << "\n";
    cout << volume(100L, 75, 15) << "\n";

    return 0;
}
```

```
// Function definitions

// Volume of cube
int volume(int s)
{
    return (s * s * s);
}

// Volume of cylinder
double volume(double r, int h)
{
    return (3.14159 * r * r * h);
}

// Volume of rectangular box
long volume(long l, int b, int h)
{
    return (l * b * h);
}
```

Compile-Time Polymorphism

II. Operator Overloading

- Operator overloading allows operators like +, -, and * to work with user-defined data types such as classes.
- It allows programmers to define the behavior of existing operators when they are applied to user-defined data types or objects.

For example:

  - The + operator adds two integers.
  - The same + operator can concatenate two strings.
  - We can also define how + should behave for custom classes.

- This makes code more readable and natural.

Compile-Time Polymorphism

II. Operator Overloading

C++ Operator Overloading

Operators That Can Be Overloaded

Operator Category	Operators
Arithmetic	+ , - , * , / , %
Bit-wise	& , | , ~ , ^ , << , >>
Logical	&& , || , !
Relational	> , < , >= , <= , == , !=
Assignment or Initialization	=
Arithmetic Assignment	+= , -= , *= , /= , %=
Shift	<< , >>
Unary	++ , --
Subscript	[ ]
Function Call	( )
Dereferencing	->
Unary Sign Prefix	+ , -
Allocate and Free	new , delete

Operators That Cannot Be Overloaded

Operator Category	Operators
Scope Resolution	::
Member Access	.
Pointer to Member Access	.*
Conditional (Ternary)	?:
Size Of	sizeof
Type Information	typeid
Alignment	alignof
Exception Specification	noexcept



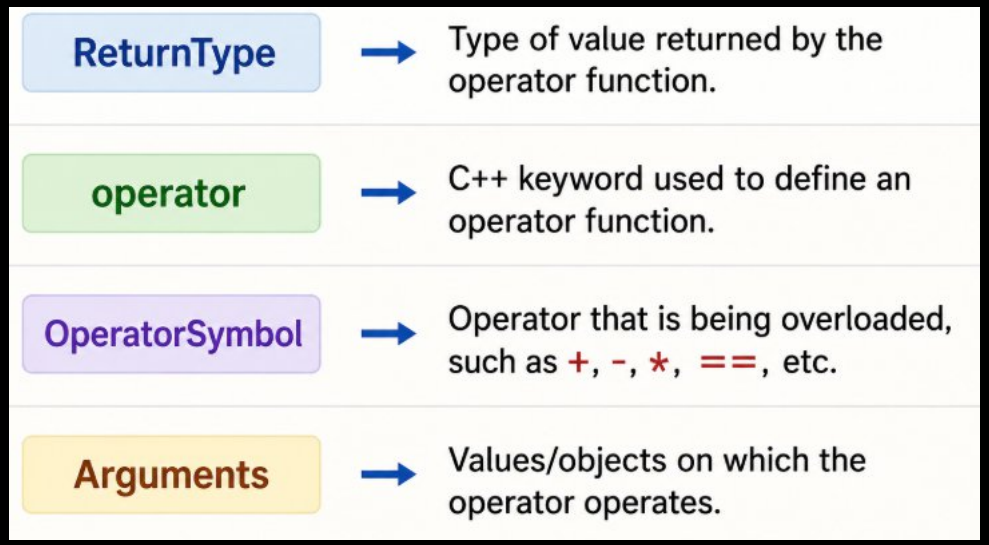
Note:
The operators listed above have fixed meaning in C++ and are essential for the compiler and language structure; therefore, they cannot be overloaded.

Compile-Time Polymorphism

II. Operator Overloading

Syntax:

```
ReturnType operator OperatorSymbol ([arg1, [arg2]])
{
    // body of operator function
}
```



Overloading of Unary Operator

- Operator overloading does not change the operator itself. It defines what that operator means when used with a particular class object.
- Giving special meaning to an operator is called operator overloading.
- Unary operator works only on the single operand.
- -,+,++,-- those are unary operator which we can overloaded
- Unary operator can be overloaded in two ways
  - Using non-static function with no argument.
  - Using friend function with one argument where the argument must be either an object of the class or an reference to an object of the class.

Overloading of Unary Operator

- Similar to other member functions, an overloaded operator function can be defined in **two ways**:
  - **Inside the class**
  - **Outside the class**

Overloading of Unary Operator

Definition Inside the Class

- The operator function is defined directly within the class body.
- It is treated as an **inline member function**.

Syntax:

```
class ClassName
{
public:
    return_type operator symbol()
    {
        // statements
    }
};
```

Overloading of Unary Operator

Definition Outside the Class

- The operator function is first **declared inside the class**.
- Its complete definition is then written **outside the class**.
- The **scope resolution operator ::** is used to specify the class to which the operator function belongs.

Syntax:

```
class ClassName
{
public:
   return_type operator symbol();
};

return_type ClassName::operator symbol()
{
   // statements
}
```

Overloading Unary Minus

```
#include <iostream>
using namespace std;

class space
{
    int x;
    int y;
    int z;

public:
    void getdata(int a, int b, int c);
    void display(void);
    void operator-(); // Overload unary minus
};
```

```
void space :: getdata(int a, int b, int c)
{
    x = a;
    y = b;
    z = c;
}
void space :: display(void)
{
    cout << x << " ";
    cout << y << " ";
    cout << z << "\n";
}
```

```
void space :: operator-()
{
    x = -x;
    y = -y;
    z = -z;
}

int main()
{
    space S;
    S.getdata(10, -20, 30);
    cout << "S : ";
    S.display();

    -S; // Calls operator-()

    cout << "S : ";
    S.display();

    return 0;
}
```

Overloading of postfix ++ operator

```
#include <iostream>
using namespace std;

class Index
{
private:
    int value;

public:
    // No-argument constructor
    Index()
    {
        value = 0;
    }
    // Constructor with one argument
    Index(int val)
    {
        value = val;
    }

    // Access function
    int GetIndex()
    {
        return value;
    }
```

```
// Overloading postfix ++ operator
    Index operator++(int)
    {

        value = value + 1;
        return Index(value);
    }
};

int main()
{
    Index idx1, idx2;

    cout << "\nIndex1 = " << idx1.GetIndex();
    cout << "\nIndex2 = " << idx2.GetIndex();

    idx1 = idx2++;   // returned object is assigned to idx1

    idx2++;         // returned object is unused

    cout << "\nIndex1 = " << idx1.GetIndex();
    cout << "\nIndex2 = " << idx2.GetIndex();

    return 0;
}
```

Limitations of Increment/Decrement Operators

- Overloaded ++ and -- do not automatically distinguish or implement prefix/postfix behavior, the programmer must explicitly define it.
- For user-defined data types, prefix and postfix ++ need to be distinguished while overloading.
- Postfix operator syntax:

  operator++(int)

Prefix operator syntax:

  operator++()

- The int parameter is a dummy parameter used only to distinguish postfix from prefix.

Overloading Binary Operator

- Binary operator are those operator which works on two operands. Eg. +, -,*,/ etc
- Binary operator acts on two operands and can be overloaded in two ways:
  - Using non-static function with single argument.
  - Using friend function with two argument where the argument must be either an object of the class or an reference to an object of the class.

Overloading Binary Operator

Syntax

```
ReturnType operator OperatorSymbol (arg)
{
   // body of operator function
}
```

OR

```
friend ReturnType operator OperatorSymbol (arg1, arg2)
{
   // body of operator function
}
```

Overloading + Operator

```
#include <iostream>
using namespace std;

class complex {
    float x; // real part
    float y; // imaginary part

public:
    complex() {} // constructor
    complex(float real, float imag) { // constructor
        x = real;
        y = imag;
    }
    complex operator+(complex);
    void display(void);
};

complex complex :: operator+(complex c) {
    complex temp; // temporary
    temp.x = x + c.x; // float additions
    temp.y = y + c.y;
    return (temp);
}
```

```
void complex :: display(void) {
    cout << x << " + j" << y << "\n";
}

int main() {
    complex C1, C2, C3;
    C1 = complex(2.5, 3.5); // invokes constructor
    C2 = complex(1.6, 2.7); // invokes constructor
    C3 = C1 + C2;           // invokes operator+

    cout << "C1 = "; C1.display();
    cout << "C2 = "; C2.display();
    cout << "C3 = "; C3.display();

    return 0;
}
```

Overloading using friend function

- Friend functions are useful in operator overloading, especially for stream operators << and >>.
- Friend functions can overload both Unary operators and Binary operators two operands
- Main difference between member function and friend function is that , the friend function requires the arguments to be explicitly passed to the function whereas the member function considers the first argument implicitly.

Overloading using friend function

Syntax: Unary

```
friend ReturnType operator OperatorSymbol (arg1 )
{
   // body of operator friend function
}
```

Syntax: Binary

```
friend ReturnType operator OperatorSymbol (arg1 , arg2)
{
   // body of operator friend function
}
```

Binary Operator overloading using friend function

```
#include <iostream>
using namespace std;

class Number
{
    int x;

public:
    Number(int a)
    {
        x = a;
    }
    // Friend function declaration
    friend Number operator +(Number n1, Number n2);

    void display()
    {
        cout << "Sum = " << x << endl;
    }
};

// Friend function definition
Number operator +(Number n1, Number n2)
{
    Number temp(0);
    temp.x = n1.x + n2.x;
    return temp;
}
```

```
int main()
{
    Number n1(10);
    Number n2(20);

    Number n3 = n1 + n2;

    n3.display();

    return 0;
}
```

Overloading Stream Operators using Friend Function

- C++ provides **iostream** for input and output operations. The extraction operator >> is used for performing input operations in the iostream library. The insertion operator << is used for performing output operations in the iostream library.
- the user-defined objects can also be read or displayed using these operators.
- For overloading <<, ostream& is taken as the first argument because cout is the first operand.
- For overloading >>, istream& is taken as the first argument.

Overloading Stream Operators using Friend Function

- Syntax (Output operator <<) :

```
friend ostream& operator<<(ostream& out, arg)
 {
//display attributes of user defined object:arg with cout or Out
Return Out;
}
```

Overloading Stream Operators using Friend Function

- Syntax (output operator >>) :

```
friend istream& operator>>(istream& in, arg)
{
//read attributes of object arg with In or cin object
Return In;
}
```

Overloading Stream Operators using Friend Function

```
#include <iostream>
using namespace std;

class Complex
{
private:
    float real;
    float imag;

public:
    Complex()
    {
    }
    Complex(float InReal)
    {
        real = InReal;
        imag = 0;
    }

    friend Complex operator+(Complex c1, Complex c2);

    friend istream& operator>>(istream& In, Complex& c);

    friend ostream& operator<<(ostream& Out, Complex& c);
};
```

Overloading Stream Operators using Friend Function

```
// Overloading +
Complex operator+(Complex c1, Complex c2)
{
    Complex c;

    c.real = c1.real + c2.real;
    c.imag = c1.imag + c2.imag;

    return c;
}

// Overloading >>
 istream& operator>>(istream& In, Complex& c)
{
    cout << "Real Part? ";
    In >> c.real;

    cout << "Imaginary Part? ";
    In >> c.imag;

    return In;
}
```

Overloading Stream Operators using Friend Function

```
// Overloading <<
ostream& operator<<(ostream& Out, Complex& c)
{
    Out << "(" << c.real;
    Out << ", " << c.imag << ")";

    return Out;
}

int main()
{
    Complex c1, c2, c3 = 3;

    cout << "Enter Complex1 c1: " << endl;
    cin >> c1;

    cout << "Enter Complex2 c2: " << endl;
    cin >> c2;

    c3 = c1 + c2;

    cout << "Result of c3 = c1 + c2: ";
    cout << c3;

    c3 = c1 + 2.0;

    cout << "\nResult of c3 = c1 + 2.0: ";
    cout << c3;

    return 0;
}
```

New and delete overloading

- Memory management deals with the allocation and deallocation of memory during program execution.
- C++ provides new and delete operators for dynamic memory management.
- new allocates memory and returns a pointer to the allocated memory.
- delete releases memory that was allocated using new.
- These operators can be overloaded to implement customized memory management.

New and delete overloading

- Overloading new Operator allows programmers to define custom memory allocation strategies. Can be used for memory pools, memory tracking, logging, and improved memory management.
- Overloading delete Operator allows programmers to define custom memory deallocation behavior. Helps in memory tracking, preventing memory leaks, and efficient resource management.

New and delete overloading

Syntax for overloading new :

```
void* operator new(size_t size)
{
    // allocate memory
    return ptr;
}
```

Syntax for overloading delete:

```
void operator delete(void* ptr)
{
    // release memory
}
```

New and delete overloading

```
#include <iostream>
using namespace std;

const int ARRAY_SIZE = 10;

class Vector
{
private:
    int *array;   // dynamically allocated array

public:
```

```
 // Overloading new operator
    void* operator new(size_t size)
    {
        Vector *my_vector;

        my_vector = ::new Vector;   // calls global
new

        my_vector->array = new int[ARRAY_SIZE];

        return my_vector;
    }
```

New and delete overloading

```
// Overloading delete operator
   void operator delete(void *vec)
   {
      Vector *my_vector;

      my_vector = (Vector*)vec;

      delete[] my_vector->array;

      ::delete my_vector;   // calls global delete
   }

   void read();
   int sum();
};
void Vector::read()
{
   for (int i = 0; i < ARRAY_SIZE; i++)
   {
      cout << "Vector[" << i << "] = ";
      cin >> array[i];
   }
}

int Vector::sum()
{
   int sum = 0;
```

New and delete overloading

```
for (int i = 0; i < ARRAY_SIZE; i++)
        sum += array[i];

    return sum;
}

int main()
{
    Vector *my_vector = new Vector;

    cout << "Enter Vector data..." <<
endl;
    my_vector->read();

    cout << "Sum of Vector = " <<
my_vector->sum();

    delete my_vector;

    return 0;
}
```

Type Conversion

- Type conversions in C++ can occur between basic types like int to int, between basic and class types like int to a class, and between class types.
- There are four main situations:

  1) Basic to basic uses implicit conversion rules.

  2) Basic to class uses constructors, like converting an int to a time class.

  3) Class to basic uses a user-defined conversion function in the class, like an overloaded typecast operator to convert a time class to an int.

  4) Class to class can use a conversion function in the source class or a constructor in the destination class, like converting a dollar class to a rupee class.

Conversion from Basic Type to Class Type

- Conversion from basic type to class type is performed using a constructor.

Syntax:

```
class ClassName
{
public:
    ClassName(BasicType variable)
    {
        // conversion
    }
};
```

```
#include <iostream>
using namespace std;

class Time
{
private:
    int hrs;
    int mins;

public:
    // Default constructor
    Time()
    {
        hrs = 0;
        mins = 0;
    }
    // Constructor for conversion from int to class type
    Time(int t)
    {
        hrs = t / 60;
        mins = t % 60;
    }

    void display()
    {
        cout << hrs << " hours ";
        cout << mins << " minutes";
    }
};

int main()
{
    int duration = 85;

    Time t = duration;   // int converted into Time object

    t.display();

    return 0;
}
```

Class type to Basic type

- **Constructor cannot perform class-to-basic type conversion.** It is performed using a **conversion/casting operator function**.
- The casting operator function **must be a member function** of the class.
- It **does not specify a return type**, not even void.
- It **takes no arguments**.

Synatx:

```
operator typename()
{
    // conversion code
}
```

Class type to Basic type

```
#include <iostream>
using namespace std;

class Time
{
private:
    int hrs;
    int mins;

public:
    // Parameterized constructor
    Time(int h, int m)
    {
        hrs = h;
        mins = m;
    }

    void display()
    {
        cout << hrs << " hours ";
        cout << mins << " minutes" << endl;
    }

    // Conversion function: Class type → int
    operator int()
    {
        return hrs * 60 + mins;
    }
};
```

Class type to Basic type

```
int main()
{
    Time t(3, 20);

    t.display();

    int duration = t;   // Time object converted to int

    cout << "Duration: " << duration << " minutes";

    return 0;
}
```

Class Type to Class Type

- Class-to-class conversion means **converting an object of one class into an object of another class**.

There are **two methods**:

1. Using a Conversion Function in the Source Class

The **source class** provides a conversion function that converts its object into the destination class.

Source class → Conversion function → Destination class

2. Using a Constructor in the Destination Class

The **destination class** provides a constructor that accepts an object of the source class.

Source object → Destination class constructor → Destination object

Class Type to Class Type

- Class-to-class conversion means **converting an object of one class into an object of another class**.

There are **two methods**:

1. Using a Conversion Function in the Source Class

The **source class** provides a conversion function that converts its object into the destination class.

Source class → Conversion function → Destination class

2. Using a Constructor in the Destination Class

The **destination class** provides a constructor that accepts an object of the source class.

Source object → Destination class constructor → Destination object

Class Type to Class Type

```
#include <iostream>
using namespace std;

class Dollar
{
public:
    int doll;

    Dollar(int x)
    {
        doll = x;
    }
    void show()
    {
        cout << "Money in dollars = " << doll << endl;
    }
};

class Rupee
{
    double rs;
```

Class Type to Class Type

```
public:
    // Constructor in destination class
    Rupee(Dollar d)
    {
        rs = d.doll * 71.59;
    }

    void show()
    {
        cout << "Money in rupees = " << rs << endl;
    }
};
```

```
int main()
{
    Dollar d1(5);

    d1.show();

    Rupee r1 = d1;   // Class-to-class conversion

    r1.show();

    return 0;
}
```

Rule for overloading operators

- Only existing operators can be overloaded. New operators cannot be created.
- The overloaded operator must have at least one operand that is of user-defined type.
- We cannot change the basic meaning of an operator. That is, we cannot redefine the plus (+) operator to subtract one value from the other.
- Overloaded operators follow the same precedence and associativity as the original operators. They cannot be changed.
- There are some operators that cannot be overloaded.

- We cannot use friend functions to overload certain operators. However, member functions can be used to overload them.
- Unary operators overloaded by means of a member function take no explicit arguments and return no explicit values, but those overloaded by means of a non-member function take one reference argument (the object of the relevant class).
- Binary operators overloaded through a member function take one explicit argument, whereas those overloaded through a non-member function take two arguments.
- When using binary operator overloading through a member function, the left-hand operand must be an object of the relevant class.
- Binary arithmetic operators such as +, -, and * must explicitly return a value. They must not attempt to change their own arguments.

- Operator that cannot be overloaded

Sizeof	Size of operator
.	Membership operator
.*	Pointer-to-member operator
:	Scope resolution operator
?:	Conditional operator

Friend function cannot be used

=	Assignment operator
()	Function call operator
[]	Subscripting operator
->	Class member access operator

Tracing Memory Leak

- A **memory leak** occurs when dynamically allocated memory is not released after it is no longer required, causing the memory to remain occupied and potentially reducing program performance or causing the program to crash.
- **Tracing memory leaks** is the process of recording dynamic memory allocations and deallocations and comparing them to identify allocated memory that has not been released.

Detecting Memory Leaks

Detecting Memory Leaks using Overloaded new and delete:

- C++ allows us to overload the new and delete operators to track and record memory allocation and deallocation, which helps in detecting memory leak.

Example:

```
#include <iostream>
#include <cstdlib>
using namespace std;

void* operator new(size_t size)
{
    void* ptr = malloc(size);

    cout << "Allocated " << size
         << " bytes at " << ptr << endl;

    return ptr;
}
void operator delete(void* ptr)
{
    cout << "Freed memory at " << ptr << endl;
    free(ptr);
}

int main()
{
    int* p = new int;
    int* arr = new int[5];

    *p = 10;

    cout << "Value = " << *p << endl;

    delete p;          // memory released

    // delete[] arr;   // intentionally omitted ? memory leak

    return 0;
}
```

Runtime Polymorphism

- Runtime polymorphism is also known as dynamic polymorphism or late binding. In this type, the function call is resolved during program execution instead of compilation. It is achieved using:

I. **Function Overriding**

- Function overriding occurs when a derived class defines one or more member functions of the base class. That base function is said to be overridden. The base class function must be declared as **virtual function** for runtime polymorphism to happen.